

name: gen-build-deb

description: PHẢI dùng khi user muốn tạo/sinh script đóng gói .deb (build-deb.sh) cho 1 project .NET/Avalonia bất kỳ, để tái sử dụng pattern đã đúc kết từ ParkingV8 (icon PNG không dùng .ico, postinst chown cả thư mục cha, wrapper LD_LIBRARY_PATH, build ở /tmp tránh lỗi DrvFs khi chạy WSL). Gọi bằng `"/gen-build-deb <đường-dẫn-hoặc-tên-project>"`. Output chính: file `scripts/linux-deb/build-deb.sh` của project đó (kèm vài file phụ trợ bắt buộc để script chạy được). KHÔNG tự chạy build/dpkg-deb — chỉ sinh file. Nếu user muốn build .deb thật ngay, dùng script vừa sinh ra, không phải chạy lại skill này.

Skill: ``/gen-build-deb`` — Sinh script đóng gói `.deb`` cho project .NET/Avalonia

Nguồn gốc: Đúc kết từ nhiều vòng fix thực tế trên ParkingV8 Avalonia (`scripts/linux-deb/build-deb.sh`) — icon không hiện do dùng `.ico` thay vì `.png`, app chậm/đơ do `postinst` chỉ `chmod` mà không `chown`, `chown` thiếu cấp thư mục cha, lỗi `dpkg-deb` khi build dir nằm trên `/mnt/<ổ Windows>` qua WSL.

>

Eval: `.claude/evals/gen-build-deb.md` — chạy thử theo CE-01/CE-02/CE-03 trước khi coi skill này APPROVED.

Input

`args` = đường dẫn hoặc tên project cần tạo `build-deb.sh` (VD: `src/SomeOtherApp` hoặc tên solution). Nếu user không chỉ rõ, hỏi lại — KHÔNG đoán project khi có nhiều lựa chọn mờ hồ.

Quy trình thực hiện (6 bước)

Bước 1 — Khảo sát project target

1. `Glob/Grep` tìm `.csproj` trong đường dẫn được chỉ định có `<OutputType>WinExe</OutputType>` hoặc `<OutputType>Exe</OutputType>` (đây là project chạy được, không phải library).
2. Nếu có nhiều `.csproj` khớp trong cùng path → hỏi user chọn đúng project chính (entry point UI), không tự đoán.
3. Đọc từ `.csproj: AssemblyName` (fallback = tên file `.csproj` không đuôi), `TargetFramework` (cảnh báo nếu `< net6.0` — self-contained `linux-x64` publish có thể không hỗ trợ).
4. Xác định `REPO_ROOT` (thư mục gốc git chứa project) và đường dẫn tương đối từ `REPO_ROOT` tới `.csproj` này → dùng làm `APP_PROJECT`.

Bước 2 — Đặt tên gói

Suy ra 3 giá trị (hỏi user xác nhận nếu tên suy ra không tự nhiên):

Biến	Cách suy ra	Ví dụ (ParkingV8)
---	---	---
PKG_NAME	kztek-<slug-lowercase-AssemblyName>	kztek-parkingv8
INSTALL_DIR	/opt/kztek/<slug>	/opt/kztek/parkingv8
Tên lệnh wrapper (/usr/bin/<cmd>)	<slug> (không dấu, không hoa)	parkingv8

Bước 3 — Tìm icon nguồn

1. Glob `"/**/*.ico"` trong project hiện tại VÀ trong project WinForms cùng solution (nếu tồn tại — pattern thường gặp: `<TênProject>/Resources/*.ico`, xem `<ApplicationIcon>` trong `.csproj` WinForms nếu có).
2. Nếu tìm được `.ico`: trích PNG frame lớn nhất có sẵn bên trong bằng Python (không cần ImageMagick/Pillow — hầu hết `.ico` hiện đại nhúng sẵn PNG cho size 256x256):

```
python
import struct
with open(ico_path, "rb") as f:
    data = f.read()
    count = struct.unpack_from("<HHH", data, 0)
    best = None
    for i in range(count):
        off = 6 + i*16
        w,h,size,offset = struct.unpack_from("<BBBBHHII", data, off)
        w, h = w or 256, h or 256
        if data[offset:offset+8] == b'\x89PNG\r\n\x1a\n' and (best is None or w > best[0]):
            best = (w, h, size, offset)
        w, h, size, offset = best
    open(png_path, "wb").write(data[offset:offset+size])
    `
```

Nếu `.ico` không có frame PNG nhúng (icon cũ dạng BMP thuần) → cảnh báo user, hỏi họ cung cấp `.png` sẵn thay thế.

3. Nếu KHÔNG tìm được icon nào → **không tự bịa icon**. Hỏi user cung cấp đường dẫn 1 file `.png`/`.svg`, hoặc xác nhận bỏ qua (`Icon=` để trống, dùng icon theme mặc định của desktop environment).
4. Icon cuối cùng LUÔN là `.png` (không dùng `.ico` làm `Icon=` trong `.desktop` — icon theme spec của Linux chỉ chắc chắn render `png/svg/xpm`).

Bước 4 — Sinh file phụ trợ (`deploy-linux/`, cạnh `.csproj` target)`

Tạo trong thư mục `deploy-linux/` cạnh `.csproj` của project:

- `run.sh` — wrapper set `LD_LIBRARY_PATH="$DIR:$LD_LIBRARY_PATH"` rồi `exec "$DIR/<AssemblyName>" "$@"`.

- `install.sh` — cài desktop entry cho user hiện tại (`~/.local/share/applications`), tạo `.<Slug>/appIcon.png` cạnh binary nếu chưa có, copy shortcut ra `~/Desktop` nếu có, `gio set metadata::trusted true` cho shortcut Desktop.
- `<pkg-slug>.desktop` — template với placeholder `__APP_DIR__`, `Icon=__APP_DIR__/.<Slug>/appIcon.png`.
- `appIcon.png` — icon đã trích ở Bước 3.

Thêm target `PublishLinuxLauncher` vào `.csproj` (copy các file trên vào `$(PublishDir)` khi `RuntimeIdentifier` bắt đầu bằng `linux`, `chmod +x` các file cần) — dùng nguyên mẫu target trong `src/ParkingV8.App/ParkingV8.App.csproj` làm khung, đổi tên file cho đúng.

Bước 5 — Sinh `scripts/linux-deb/build-deb.sh`

Dùng `scripts/linux-deb/build-deb.sh` của `ParkingV8` làm template gốc 1:1 (đường dẫn tương đối trong cùng repo này) — copy cấu trúc 7 bước, đổi các biến sau cho đúng project target, GIỮ NGUYÊN mọi comment giải thích gotcha (rất quan trọng, tránh người sau tái phạm lỗi đã fix):

Đổi trong script	Từ (ParkingV8)	Thành			
---	---	---			
<code>PKG_NAME</code>	<code>kztek-parkingv8</code>	(Bước 2)			
<code>INSTALL_DIR</code>	<code>/opt/kztek/parkingv8</code>	(Bước 2)			
<code>APP_PROJECT</code>	<code>\$REPO_ROOT/src/ParkingV8.App/ParkingV8.App.csproj</code>	(Bước 1)			
Tên binary trong wrapper/chmod	<code>ParkingV8.App</code>	<code><AssemblyName></code>			
Tên file <code>.desktop</code> , <code>Icon=</code>	<code>parkingv8.desktop, .iParkingV8/appIcon.png</code>	<code><pkg-slug>.desktop, .<Slug>/appIcon.png</code>			
<code>Depends</code> trong <code>DEBIAN/control</code>	Danh sách lib <code>ParkingV8</code> dùng (<code>FFmpeg/camera/X11...</code>)	Hỏi user hoặc giữ tối thiểu <code>libc6, libgcc-s1, libstdc++6, zlib1g, libicu70</code>	<code>libc u72</code>	<code>libc u74</code>	<code>libicu 76</code> — KHÔNG tự thêm lib cụ thể domain khác mà chưa xác nhận project mới có dùng không

Descript ion trong control	Mô tả ParkingV8	Hỏi user 1 câu mô tả ngắn, hoặc dùng tên project			
----------------------------------	-----------------	---	--	--	--

PHẢI giữ nguyên các cơ chế đã fix (không được bỏ khi tham số hoá):

1. `BUILD_DIR="/tmp/kztek-deb-build/$PKG_NAME"` — build ở `/tmp`, KHÔNG build trực tiếp trên `/mnt/<ổ Windows>` khi chạy WSL.
2. `postinst: chmod -R 777 "$INSTALL_DIR" VÀ chown -R "$TARGET_USER": "$TARGET_USER" "$INSTALL_PARENT_DIR"` (đọc `SUDO_USER`, fallback `logname`) — `chown` **thư mục cha**, không chỉ install dir.
3. Icon PNG (không `.ico`) trong `.desktop`.
4. `dpkg-deb --build --root-owner-group` rồi copy `.deb` hoàn chỉnh ra `dist/` (không build trực tiếp file `.deb` lên `/mnt/*`).

Bước 6 — Báo cáo kết quả

Output chính theo đúng yêu cầu: đường dẫn `scripts/linux-deb/build-deb.sh` vừa tạo cho project. Liệt kê kèm các file phụ trợ đã tạo/sửa (`deploy-linux/*`, `.csproj` target). Nhắc user: script chỉ chạy được trên Linux/WSL có `dotnet` SDK + `dpkg-deb`; muốn build thật thì tự chạy `bash scripts/linux-deb/build-deb.sh [version]`, skill này không tự chạy build.

Không được làm

- KHÔNG tự chạy `dotnet publish/dpkg-deb` khi user chỉ gọi `/gen-build-deb` (đó là việc của bước sau, do user chủ động chạy script).
- KHÔNG overwrite `scripts/linux-deb/build-deb.sh` của ParkingV8 (project nguồn) khi target là project khác — luôn viết vào đúng path project được chỉ định.
- KHÔNG tự bịa icon nếu không tìm được — hỏi user.
- KHÔNG bỏ bớt các comment giải thích gotcha khi copy template — người đọc sau cần biết TẠI SAO, không chỉ CÁI GÌ.